

5. Step 3 — Couche web, API REST & sécurité

Proposition de projet — Enterprise Inventory & Logistics Management System (EILMS)

Partie du document : Section 3.3 — Web Tier, REST APIs & Security

Objectif : construire un panneau d'administration MVC et sécuriser des endpoints RESTful.

Les 3 instructions du Step 3

- 1. Servlet/JSP avec authentification par session (legacy) :** comprendre les bases d'un conteneur web « à l'ancienne » avant les frameworks modernes. Une Servlet traite une requête HTTP manuellement, une JSP génère du HTML côté serveur, et l'authentification par session stocke l'état de connexion dans un objet HttpSession côté serveur.
- 2. Transition vers Spring MVC / Spring REST :** remplacer les Servlets manuelles par des contrôleurs Spring annotés (@RestController), exposant des endpoints conformes à la spécification OpenAPI (documentation d'API standardisée, utilisable par des outils comme Swagger). Exemple : @GetMapping("/items/{id}") pour récupérer un item par son id via HTTP GET.
- 3. Spring Security avec RBAC et JWT :** sécuriser les endpoints avec un contrôle d'accès basé sur les rôles (RBAC — Role-Based Access Control, ex : ADMIN peut tout faire, EMPLOYEE peut seulement consulter) et une authentification par jeton JWT (JSON Web Token) — un jeton signé envoyé dans l'en-tête Authorization: Bearer <token> de chaque requête, plutôt qu'une session stockée côté serveur.

Pourquoi ce découpage en 3 sous-étapes

Le cours fait volontairement passer par l'approche « legacy » (Servlet/JSP + session) avant l'approche moderne (Spring REST + JWT), pour bien comprendre ce que les frameworks automatisent derrière les annotations.